Annamacharya Institute of Technology & Sciences, Tirupati (AITS-T)

(Approved by AICTE New Delhi & Permanent Affiliation to JNTUA, Ananthapuramu; A five B. Tech Programmes (CSE, ECE, EEE, CE & ME) are accredited by NBA, New Delhi. Accredited by NAAC with 'A' grade Bangalore; A-grade awarded by AP Knowledge Mission; Accredited by the Institution of Engineers IE(I), Kolkata; Recognized under sections 2(f) & 12 (B) of UGC Act 1956)

Tirupati dist. – 517507, Andhra Pradesh

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

MINI PROJECT REPORT

Submitted in Partial Fulfillment of the Requirements for the Award of the Degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

ON

URBANPULSE – City Traffic Navigation System using Graph Data Structure

Submitted by

J. NAVEEN

B. TEJA, KIRAN

R. KARTHIKEYA

Academic Year: 2025 – 2026

Annamacharya Institute of Technology & Sciences, Tirupati (AITS-T)

Tirupati Dist. – 517507, Andhra Pradesh

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

DECLARATION

I hereby declare that the mini project report entitled "URBANPULSE - Intelligent Traffic Routing and Navigation Simulator" submitted by me to the Department of Computer Science and Engineering (Data Science), Annamacharya Institute of Technology & Sciences, Tirupati (AITS-T), in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology, is a record of bonafide work carried out by me.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

All the information furnished in this report is genuine and true to the best of my knowledge.

Place: Tirupati
Date: _____

J. Naveen
Roll No: 23AK1A3230
B.Tech III Year, CSE (DS)
AITS, Tirupati

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to all those who have contributed to the successful completion of this mini project. First and foremost, I am immensely thankful to God Almighty for bestowing upon me the strength, wisdom, and perseverance to complete this work.

I am deeply grateful to Dr. C. Nadhamuni Reddy, Principal, Annamacharya Institute of Technology & Sciences, Tirupati (AITS-T), for providing an excellent academic environment and world-class infrastructure that made this project a reality.

I would like to convey my heartfelt appreciation to Dr. B. Ramu, Professor and Head of Department, Computer Science and Engineering (Data Science), for his constant support, encouragement, and administrative backing throughout the project.

I also extend my sincere thanks to all the faculty members and staff of the Department of Computer Science and Engineering (Data Science) for their academic support and encouragement during the course of this project.

Finally, I express my deepest gratitude to my family and friends for their unwavering support, love, and encouragement throughout my academic journey.

J. Naveen
Roll No: 23AK1A3230

ABSTRACT

The unprecedented growth of urban populations and the consequent escalation of vehicular traffic have posed severe challenges to modern city transportation systems. Conventional navigation tools are increasingly inadequate in addressing the dynamic nature of real-world traffic, where conditions change rapidly due to accidents, construction, weather, and peak-hour congestion. The need for intelligent, responsive, and algorithm-driven routing systems has therefore become a critical area of research and application in the field of Smart City infrastructure.

UrbanPulse is a full-stack web-based intelligent traffic routing and navigation simulator developed to model the operational principles of modern navigation systems such as Google Maps and HERE Maps. The application constructs a city road network in the form of a weighted graph, where intersections are represented as nodes and roads as weighted edges reflecting distance, speed limits, and traffic density. The system employs graph traversal and pathfinding algorithms — specifically Dijkstra's Algorithm and the A* Search Algorithm — to determine optimal routes between any two user-selected points in the network.

A distinguishing feature of UrbanPulse is its traffic-aware routing engine, which dynamically adjusts edge weights in response to simulated congestion levels. The system supports real-time simulation of traffic events including vehicle movement, traffic signal cycling, road accidents, and construction delays. A comprehensive congestion analytics dashboard presents performance metrics and route comparison data, enabling users to observe and analyze the behavioral differences between static shortest-path algorithms and traffic-weighted pathfinding.

The application is built using the MERN (MongoDB, Express.js, React.js, Node.js) stack, with an interactive front-end visualization layer leveraging HTML5 Canvas and Leaflet.js with OpenStreetMap integration. The result is an educational and practically demonstrable system that bridges the gap between theoretical graph algorithm study and real-world navigation engineering.

Keywords: Traffic Routing, Dijkstra Algorithm, A Search, Graph Theory, Navigation Simulator, Congestion Analysis, Smart City, MERN Stack, Route Optimization.*

TABLE OF CONTENTS

Declaration ii

Acknowledgement iii

Abstract iv

Table of Contents v

List of Figures vi

List of Tables vii

Chapter 1 – Introduction 1

1.1 Background 1
1.2 Need for the Project 2
1.3 Problem Statement 3
1.4 Objectives 3
1.5 Scope of the Project 4
1.6 Applications 4

Chapter 2 – Literature Survey 5

2.1 Existing Systems 5
2.2 Limitations of Existing Systems 6
2.3 Proposed System 7
2.4 Advantages of Proposed System 7
2.5 Comparative Analysis 8

Chapter 3 – System Analysis 9

3.1 Functional Requirements 9
3.2 Non-Functional Requirements 10
3.3 Feasibility Study 10
3.4 Use Case Description 11

Chapter 4 – System Design 12

4.1 System Architecture 12
4.2 Module Design 12
4.3 Data Flow Diagrams 13
4.4 UML Use Case Diagram 14
4.5 UML Class Diagram 14
4.6 Sequence Diagram 15

Chapter 5 – Implementation 16

5.1 Frontend Implementation 16
5.2 Backend Implementation 17
5.3 Graph Data Structures 17
5.4 Dijkstra Algorithm 18
5.5 A* Search Algorithm 19
5.6 Traffic-Aware Routing Logic 20

5.7 MST Implementation 21
5.8 Route Optimization Workflow 21
Chapter 6 – Results and Discussion 22
Chapter 7 – Testing 24
Chapter 8 – Conclusion and Future Enhancements 26
References 28
Appendices 30

LIST OF FIGURES

Fig 1.1 Typical Urban Traffic Congestion Scenario2
Fig 1.2 Navigation System Architecture Overview3
Fig 2.1 Comparison of Existing Navigation Systems6
Fig 3.1 Use Case Diagram – UrbanPulse System11
Fig 4.1 System Architecture – UrbanPulse12
Fig 4.2 DFD Level 0 – Context Diagram13
Fig 4.3 DFD Level 1 – System Processes13
Fig 4.4 UML Class Diagram14
Fig 4.5 Sequence Diagram – Route Calculation15
Fig 5.1 Graph Data Structure Representation17
Fig 5.2 Dijkstra Algorithm Execution Trace18
Fig 5.3 A* Algorithm Heuristic Visualization19
Fig 5.4 Traffic-Aware Edge Weight Adjustment20
Fig 6.1 Home Dashboard – UrbanPulse Application22
Fig 6.2 City Road Network Visualization22
Fig 6.3 Route Solver – Origin/Destination Selection23
Fig 6.4 Computed Route Highlighted on Map23
Fig 6.5 Traffic Simulation – Live Congestion View23
Fig 6.6 Analytics Dashboard – Performance Metrics24

LIST OF TABLES

Table 2.1 Comparative Analysis of Navigation Systems8
Table 3.1 Functional Requirements9
Table 3.2 Non-Functional Requirements10

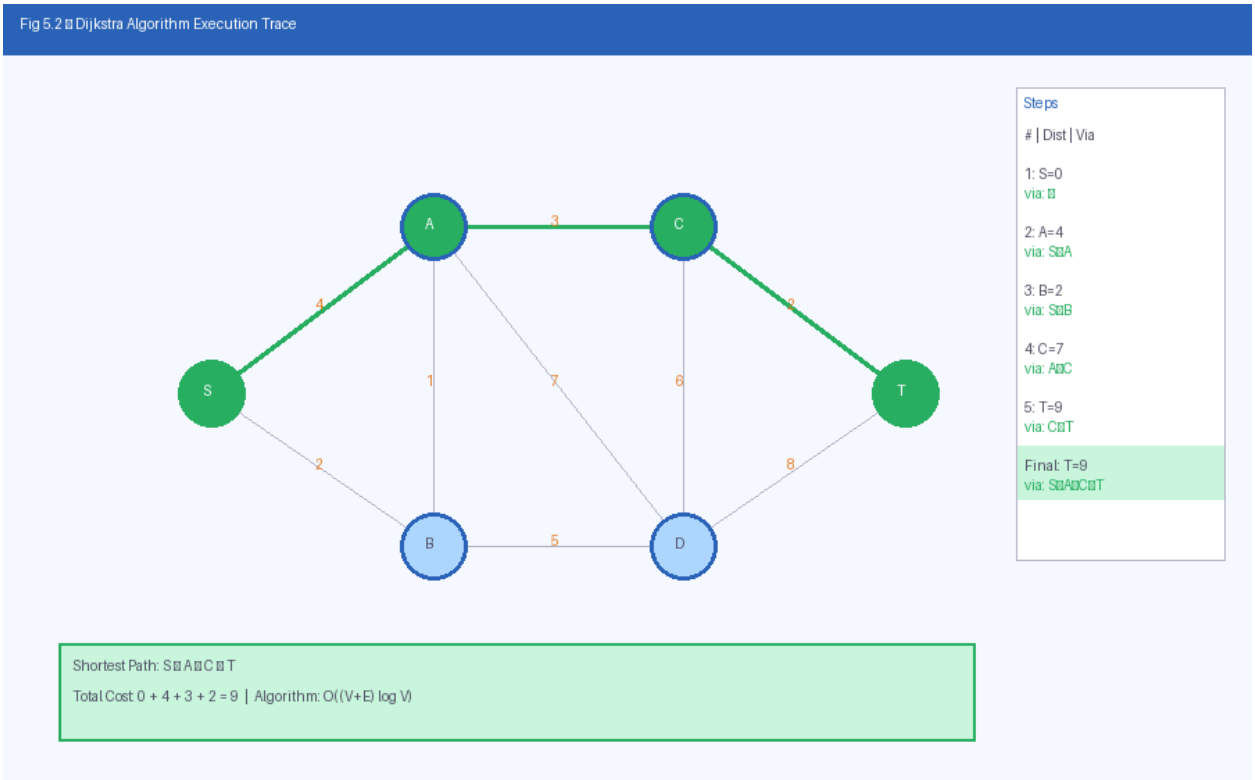


Table 5.1 Dijkstra Algorithm – Step-by-Step Example18

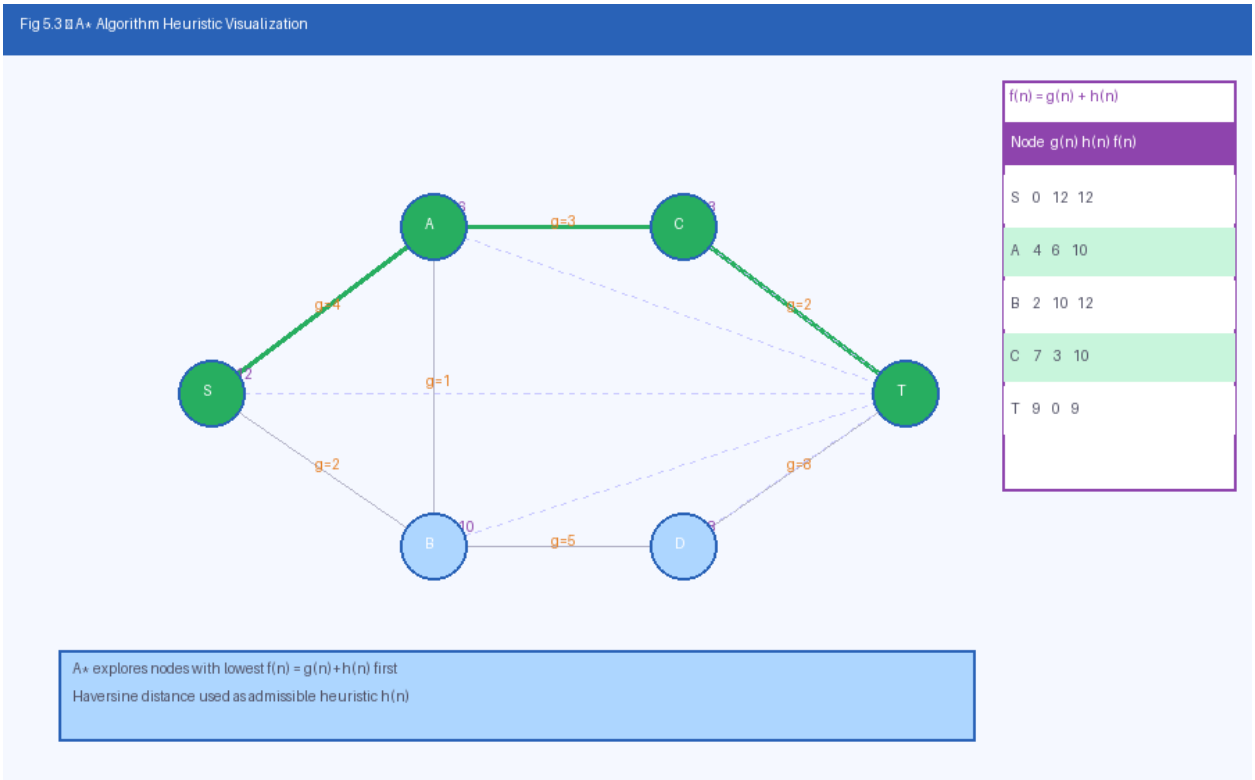


Table 5.2 A* vs Dijkstra – Feature Comparison19
Table 6.1 Route Performance Metrics23
Table 6.2 Dijkstra vs Traffic-Aware Routing Comparison24
Table 7.1 Functional Test Cases24
Table 7.2 Integration Test Cases25
Table 7.3 Performance Test Results26
Table A.1 Software Requirements30
Table B.1 Hardware Requirements30

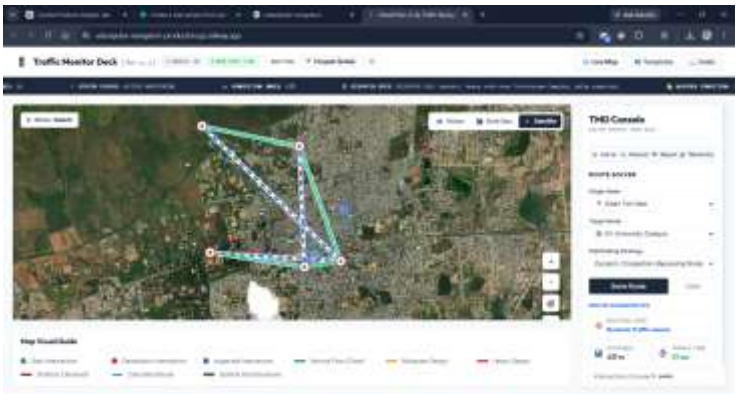
CHAPTER 1: INTRODUCTION

1.1 Background

The twenty-first century has witnessed an unprecedented surge in urbanization, with more than 56% of the world's population residing in cities as of 2023, a figure projected to reach 68% by 2050 according to United Nations estimates. This demographic shift has placed enormous strain on urban transportation infrastructure, leading to chronic traffic congestion, increased travel times, environmental degradation from vehicular emissions, and significant economic losses due to productivity decline. Cities such as Bengaluru, Mumbai, Jakarta, and Los Angeles consistently rank among the most traffic-congested urban centres globally, reflecting a worldwide crisis in transportation management.

In response to these challenges, the field of Intelligent Transportation Systems (ITS) has emerged as a critical discipline that leverages computational techniques, sensor networks, and communication technologies to optimize urban mobility. Central to ITS is the concept of intelligent navigation — systems capable of dynamically determining optimal routes in real time by considering not merely static road distances, but also live traffic density, road closures, signal timing, and other contextual factors. Companies such as Google, Uber, HERE Technologies, and TomTom have invested billions of dollars in developing navigation platforms that process vast quantities of real-time data to provide drivers with accurate and efficient routing.

At the computational core of these navigation systems lie graph algorithms — mathematical constructs originally developed in the field of discrete mathematics and computer science. A city's road network can naturally be modelled as a weighted directed graph, where intersections correspond to vertices, roads correspond to edges, and weights represent travel cost metrics such as distance, travel time, or congestion index. Shortest-path algorithms, including Dijkstra's algorithm and the A* (A-Star) search algorithm, form the backbone of route computation in such graph-based models.



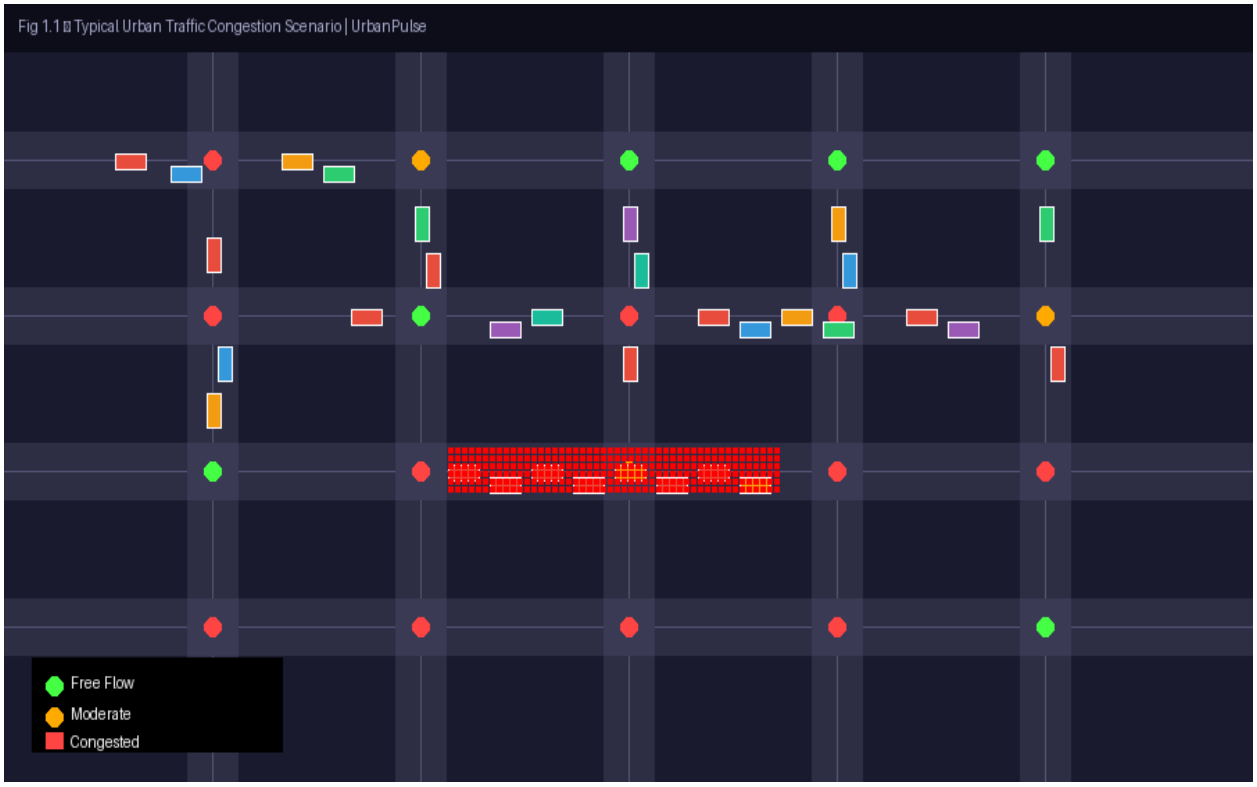


Figure: Typical Urban Traffic Congestion Scenario

1.2 Need for the Project

Despite the widespread availability of commercial navigation applications, a significant gap exists between theoretical understanding of graph algorithms taught in academic curricula and their practical application in real-world navigation systems. Students of Computer Science and Data Science routinely study Dijkstra's algorithm and graph theory in isolation, without a practical platform that demonstrates how these algorithms interact with dynamic, real-world data to produce routing decisions.

Furthermore, existing commercial platforms are black-box systems — they conceal their internal computational processes from users. There is no publicly available, open educational tool that allows students, researchers, and urban planners to observe, analyse, and compare the behaviour of different routing algorithms under varying traffic conditions in a controlled, interactive environment.

UrbanPulse addresses this need by providing a transparent, interactive simulation platform that visualizes the internal workings of traffic routing algorithms. It serves as both an educational tool and a demonstrable prototype of the computational principles underlying modern navigation systems.

1.3 Problem Statement

Existing navigation systems, while functionally sophisticated, operate as closed, proprietary platforms that provide no educational visibility into routing logic or algorithm

behaviour. Academic study of graph algorithms, on the other hand, is typically limited to theoretical exposition without practical, interactive implementation. There is a need for an open, configurable, web-based traffic routing simulator that:

- Models a city's road network as a graph with realistic properties.
- Implements and visualizes shortest-path and traffic-aware routing algorithms.
- Simulates dynamic traffic conditions including congestion, accidents, and signal timing.
- Enables users to compare routing decisions under different algorithmic strategies.
- Presents performance analytics in an accessible dashboard format.

1.4 Objectives

The primary objectives of the UrbanPulse project are as follows:

1. To design and implement a city road network using weighted graph data structures suitable for routing algorithm application.
2. To implement Dijkstra's Algorithm and the A* Search Algorithm for optimal path computation across the city network.
3. To develop a traffic simulation engine capable of generating dynamic congestion, vehicle flow, signal cycling, and event-based disruptions.
4. To design a traffic-aware routing mechanism that dynamically adjusts route recommendations in response to simulated congestion data.
5. To build an interactive web-based visualization layer that renders the city graph, computed routes, and traffic overlays in real time.
6. To develop a congestion analytics dashboard providing performance comparison between routing strategies.

1.5 Scope of the Project

The scope of UrbanPulse encompasses the following functional and technical boundaries:

- Construction of a configurable city road network with multiple intersections and road segments.
- Implementation of at least two pathfinding algorithms with visual step-by-step execution tracing.
- Simulation of time-varying traffic conditions including vehicular movement and density fluctuations.
- Event-driven simulation of accidents, road closures, and construction delays.
- Full-stack web application development with a React.js frontend and Node.js/Express.js backend.
- Integration with Leaflet.js and OpenStreetMap for geospatial road network visualization.

- Exclusion of real-time GPS data integration and actual live traffic feeds (simulated data only).

1.6 Applications

The UrbanPulse simulator has potential applications across multiple domains:

- Academic Education: Serves as an interactive teaching aid for graph algorithms, data structures, and network theory courses.
- Urban Planning: Allows city planners to model and evaluate the impact of road network modifications on traffic flow.
- Research Prototype: Functions as a testbed for experimenting with novel routing heuristics and traffic management strategies.
- Navigation System Development: Provides a proof-of-concept platform for developers building lightweight navigation applications.
- Traffic Engineering Simulations: Enables traffic engineers to study congestion propagation and bottleneck identification in road networks.

CHAPTER 2: LITERATURE SURVEY

2.1 Existing Systems

Several traffic routing and navigation systems have been developed at both commercial and academic levels. A comprehensive review of these systems provides insight into the state of the art and the specific gaps that UrbanPulse aims to address.

2.1.1 Google Maps Navigation

Google Maps is the world's most widely used navigation platform, offering real-time traffic data, turn-by-turn navigation, and multi-modal route planning across walking, driving, cycling, and public transit. The platform incorporates satellite imagery, Street View, and crowd-sourced traffic data from millions of Android devices. Its routing engine employs highly optimized variants of Dijkstra's and A* algorithms enhanced with hierarchical decomposition techniques such as Contraction Hierarchies (CH) to achieve millisecond-scale query performance on continental-scale road networks. However, Google Maps operates entirely as a proprietary closed system, offering no visibility into its routing logic.

2.1.2 OpenStreetMap and OSRM

OpenStreetMap (OSM) is a collaborative, open-source geographic database that provides freely accessible road network data for the entire globe. The Open Source Routing Machine (OSRM) is a high-performance routing engine built atop OSM data, capable of computing shortest routes across continental networks in under 10 milliseconds using Contraction Hierarchies. OSRM is widely used in research and open-source navigation applications. While OSRM provides accessible routing functionality, it does not include a built-in traffic simulation layer or interactive algorithm visualization.

2.1.3 SUMO – Simulation of Urban Mobility

SUMO is an open-source, microscopic traffic simulation package developed by the German Aerospace Center (DLR). It models individual vehicle behaviour, traffic signal control, pedestrian movement, and public transport operations with high fidelity. SUMO is extensively used in academic research for studying traffic flow dynamics, signal optimization, and autonomous vehicle interaction. However, SUMO's interface is complex, requires specialized configuration in XML, and does not include web-based interactive routing or algorithm visualization.

2.1.4 PathFinding.js

PathFinding.js is a JavaScript library providing implementations of multiple pathfinding algorithms including A*, Dijkstra, BFS, DFS, and others, primarily targeted at 2D grid-based map navigation used in gaming applications. While it offers useful algorithm

visualization capabilities in grid format, it is not designed for geospatial road network simulation, traffic modeling, or realistic city navigation scenarios.

2.1.5 Waze

Waze is a community-driven navigation application acquired by Google that specializes in real-time traffic alerts sourced from its user community. Its routing engine factors in incident reports, police presence, hazards, and real-time congestion data. Like Google Maps, Waze operates as a closed platform with no programmatic access to its routing logic or algorithm internals.

2.2 Limitations of Existing Systems

System	Primary Limitation	Open Source	Algorithm Visibility	Traffic Simulation
Google Maps	Closed proprietary platform; no algorithm visibility	No	None	Live (hidden)
OSRM	No traffic simulation or interactive visualization	Yes	Partial	None
SUMO	Complex setup; no web interface; no routing UI	Yes	Partial	Advanced
PathFinding.js	Grid-based only; no geospatial or traffic support	Yes	Full	None
Waze	Closed proprietary; community-driven, not algorithmic	No	None	Live (hidden)
HERE Maps API	Enterprise cost; limited educational access	No	None	Limited

2.3 Proposed System

UrbanPulse is proposed as an open, web-based, interactive traffic routing simulator that uniquely combines the following capabilities in a single integrated platform:

- A geospatially-aware city road network modelled as a weighted graph.
- Transparent implementation and step-by-step visual execution of Dijkstra and A* algorithms.
- A dynamic traffic simulation engine generating realistic congestion patterns and events.
- A traffic-aware routing engine that adapts to simulated traffic conditions.
- An interactive, browser-based visualization interface accessible without specialized software installation.
- A congestion analytics dashboard providing performance metrics and algorithm comparisons.

2.4 Advantages of Proposed System

- Transparency: Users can observe the internal routing logic in real time, unlike black-box commercial systems.
- Accessibility: Fully browser-based with no installation requirements beyond a modern web browser.
- Educational Value: Bridges the gap between theoretical algorithm study and practical navigation engineering.
- Extensibility: Modular architecture allows addition of new algorithms and city maps without restructuring the codebase.
- Cost-Free: No API subscription costs; uses OpenStreetMap open data.
- Comparative Analysis: Enables direct side-by-side comparison of routing strategies under identical traffic conditions.

2.5 Comparative Analysis

Feature	Google Maps	OSRM	SUMO	PathFinding.js	UrbanPulse
Web-Based Interface	Yes	Partial	No	Yes	Yes
Geospatial Map	Yes	Yes	Yes	No	Yes
Traffic Simulation	Live	No	Advanced	No	Simulated
Algorithm Visualization	No	No	No	Yes	Yes
Open Source	No	Yes	Yes	Yes	Yes
Multiple Algorithms	No	No	No	Yes	Yes
Analytics Dashboard	Basic	No	Advanced	No	Yes
Educational Purpose	No	Partial	Partial	Yes	Yes
Traffic-Aware Routing	Yes	Limited	Yes	No	Yes

CHAPTER 3: SYSTEM ANALYSIS

3.1 Functional Requirements

Functional requirements define the specific behaviours and operations the UrbanPulse system must be capable of performing.

FR ID	Requirement	Priority	Module
FR-01	System shall load and render a city road network as an interactive graph	High	City Network
FR-02	User shall select origin and destination nodes on the map	High	Route Optimization
FR-03	System shall compute the shortest path using Dijkstra's Algorithm	High	Algorithm Engine
FR-04	System shall compute an optimal path using the A* Algorithm	High	Algorithm Engine
FR-05	System shall highlight the computed route on the map visualization	High	Visualization Layer
FR-06	System shall simulate vehicular traffic movement on road segments	Medium	Traffic Simulation
FR-07	System shall simulate traffic signal cycling at intersections	Medium	Traffic Simulation
FR-08	System shall simulate road events such as accidents and construction	Medium	Traffic Simulation
FR-09	System shall adjust edge weights dynamically based on congestion	High	Traffic-Aware Routing
FR-10	System shall display travel time, distance, and congestion metrics	High	Analytics Dashboard
FR-11	System shall generate and display a Minimum Spanning Tree of the road network	Low	Algorithm Engine
FR-12	System shall allow users to compare Dijkstra vs traffic-aware routing results	Medium	Analytics Dashboard

3.2 Non-Functional Requirements

NFR ID	Requirement	Metric
NFR-01	Performance: Route calculation shall complete within 2 seconds for networks up to 500 nodes	Response Time

NFR ID	Requirement	Metric
NFR-02	Scalability: System shall support up to 1,000 concurrent users without performance degradation	Load Capacity
NFR-03	Usability: UI shall be navigable by a first-time user without prior instruction	Usability Score
NFR-04	Reliability: System uptime shall be at least 99% during academic demonstration periods	Availability
NFR-05	Maintainability: Codebase shall follow modular architecture with documented API endpoints	Code Quality
NFR-06	Security: User inputs shall be validated and sanitized to prevent injection attacks	Security Posture
NFR-07	Portability: Application shall function correctly on Chrome, Firefox, and Safari	Browser Compatibility

3.3 Feasibility Study

3.3.1 Technical Feasibility

The project is technically feasible. All required technologies — React.js, Node.js, Express.js, Leaflet.js, and HTML5 Canvas — are mature, well-documented, and freely available open-source frameworks. Graph algorithms including Dijkstra and A* are well-understood and have established JavaScript implementations. The deployment platform (Railway.app) provides containerized Node.js hosting with zero-configuration deployment.

3.3.2 Operational Feasibility

The web-based nature of UrbanPulse ensures high operational feasibility. End users require only a modern web browser to access and operate the system, eliminating software installation barriers. The interface is designed to be intuitive for computer science students familiar with web applications.

3.3.3 Economic Feasibility

The project incurs no software licensing costs, utilizing exclusively open-source technologies and open geographic data. Cloud hosting costs are minimal for academic demonstration purposes. The project is therefore highly economically feasible within a student project budget.

3.4 Use Case Description

The UrbanPulse system involves two primary actors: the End User (student, researcher, or urban planner) and the System (the automated traffic simulation and routing engine).

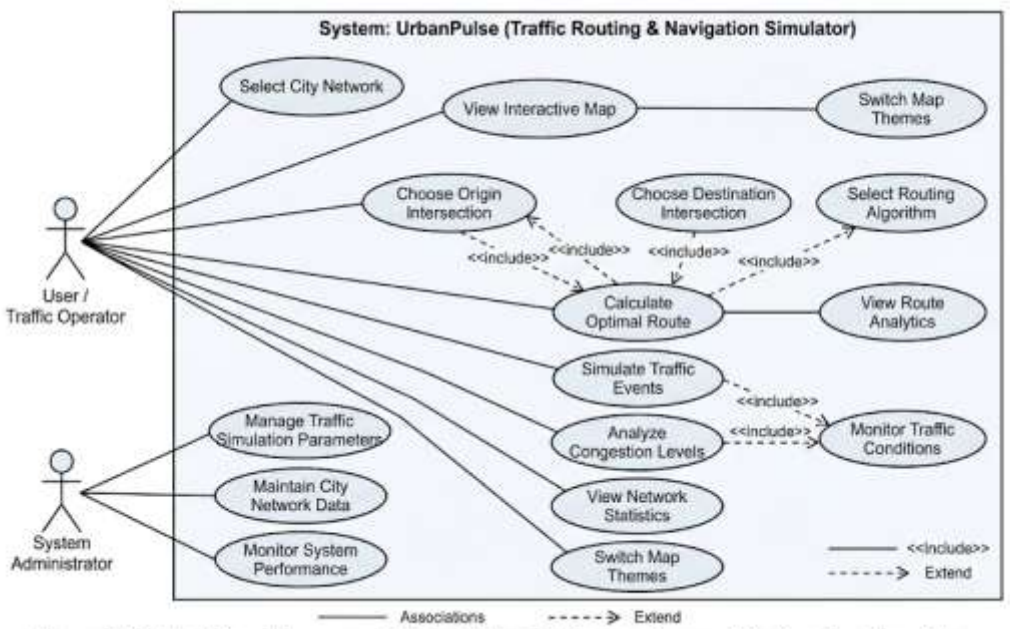


Figure X.X: Use Case Diagram of UrbanPulse Traffic Routing and Navigation Simulator

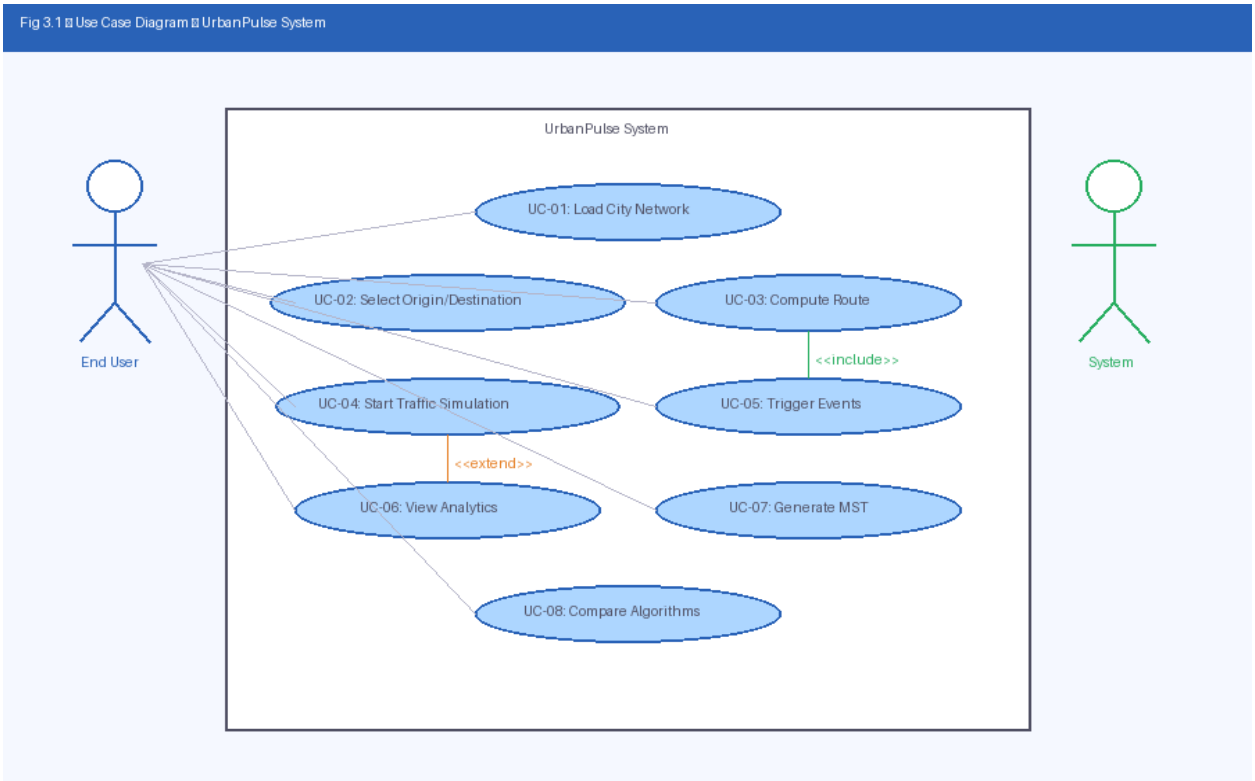


Figure: Use Case Diagram – UrbanPulse System

Use Case	Actor	Description	Precondition
UC-01: Load City Network	User	User selects a city map to load the road network graph	System is running
UC-02: Select Route Endpoints	User	User clicks on map to designate origin and destination nodes	City network loaded

Use Case	Actor	Description	Precondition
UC-03: Compute Shortest Path	User/System	System applies selected algorithm and returns optimal route	Endpoints selected
UC-04: Start Traffic Simulation	User	User initiates real-time traffic flow simulation	City network loaded
UC-05: Trigger Traffic Event	User	User simulates accident or construction on a road segment	Simulation running
UC-06: View Analytics	User	User reviews route metrics and algorithm comparison data	Route computed
UC-07: Generate MST	User	User requests Minimum Spanning Tree of the road network	City network loaded

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

UrbanPulse follows a three-tier client-server architecture comprising a Presentation Tier (React.js frontend), an Application Tier (Node.js/Express.js backend), and a Data Tier (in-memory graph data structures with optional persistent storage). Communication between the frontend and backend is facilitated through a RESTful API layer over HTTP/HTTPS, with real-time updates managed via WebSocket connections for live traffic simulation data.

The frontend interacts with the Leaflet.js mapping library for geospatial rendering and utilizes HTML5 Canvas for custom overlay rendering of graph nodes, edges, computed routes, and traffic density indicators. The backend hosts the core graph data model and exposes API endpoints for route computation, traffic simulation control, and analytics data retrieval.

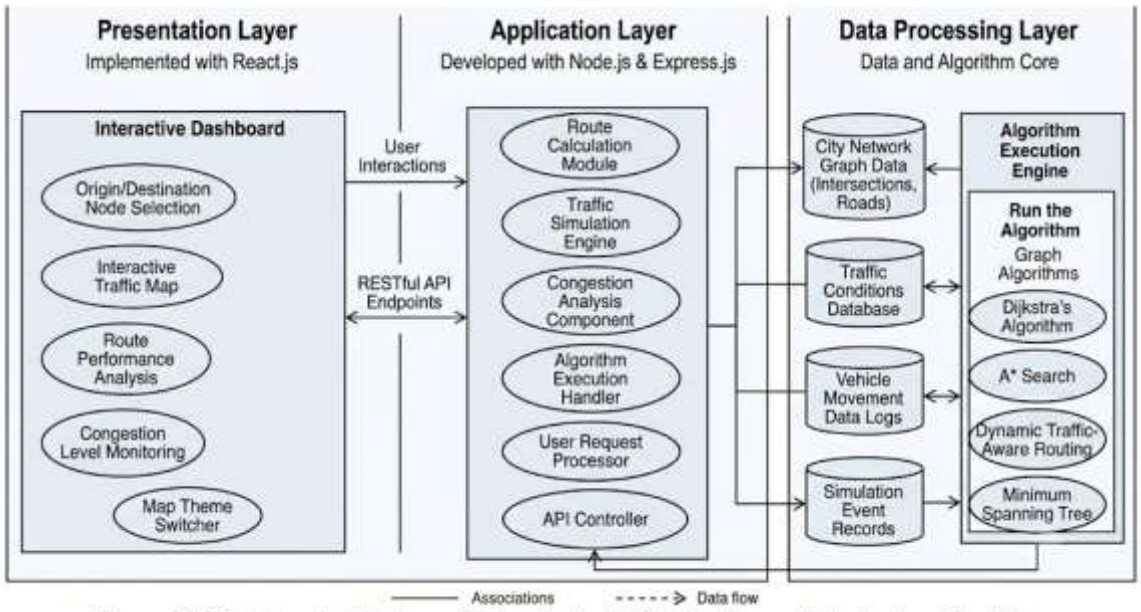


Figure X.X: System Architecture of UrbanPulse Traffic Routing and Navigation Simulator

This modular architecture ensures efficient computational throughput for route calculation and traffic modeling while maintaining strict decoupling between the visualization, processing, and simulation modules.

Figure: System Architecture of UrbanPulse Traffic Routing and Navigation Simulator

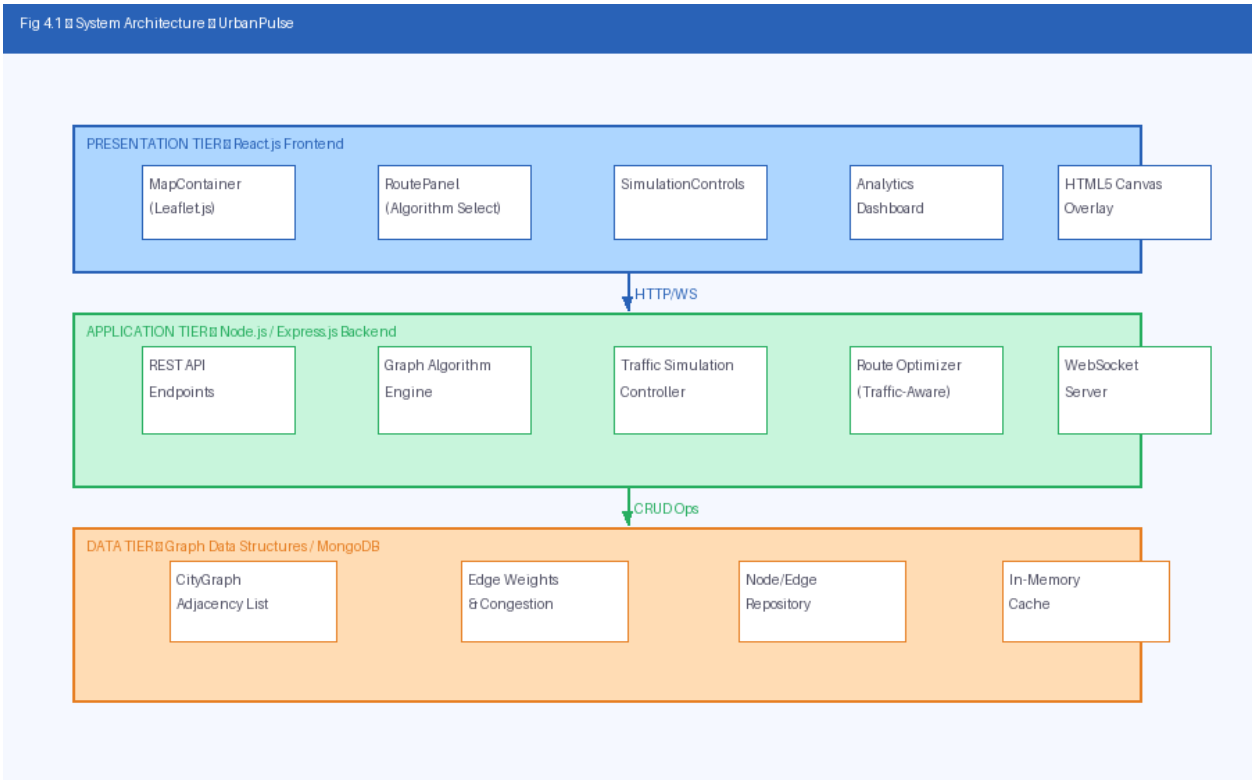


Figure: System Architecture – UrbanPulse

4.2 Module Design

Module 1: City Network Construction

This module is responsible for defining the city road network as a weighted directed graph. Each node represents an intersection with attributes including a unique identifier, geographic coordinates (latitude/longitude), and a display label. Each edge represents a road segment with attributes including source node, destination node, distance weight, speed limit, and a dynamic congestion factor. The module exposes methods for graph construction, node/edge retrieval, and adjacency list generation.

Module 2: Traffic Simulation Engine

The Traffic Simulation Engine manages the dynamic state of the city road network, simulating vehicular flow, traffic signal cycling, and random traffic events. It operates on a time-stepped simulation loop, updating edge congestion weights at configurable intervals. The module generates traffic density values for each edge, scales them to visual indicators (green/yellow/red), and broadcasts updates to connected frontend clients via WebSocket events.

Module 3: Graph Algorithm Processing Engine

This module contains the implementations of all pathfinding and graph analysis algorithms. It receives a graph snapshot and a source-destination pair, executes the specified algorithm, and returns the computed path, total cost, and the algorithm's execution trace for

visualization purposes. Supported algorithms include Dijkstra's Algorithm, A* Search, and Kruskal's/Prim's MST algorithm.

Module 4: Route Optimization System

The Route Optimization System interfaces between the Graph Algorithm Engine and the Traffic Simulation Engine to produce traffic-aware routing recommendations. When traffic-aware mode is activated, this module modifies edge weights using the formula: adjusted_weight = base_distance × (1 + congestion_factor), where congestion_factor ranges from 0.0 (free flow) to 2.0 (severe congestion). The adjusted graph is then passed to the pathfinding algorithm for route computation.

Module 5: Traffic Analytics Dashboard

This module aggregates route performance data including total distance, estimated travel time, number of intersections traversed, average congestion level along the route, and comparative metrics between standard shortest-path and traffic-aware routing. Data is formatted and transmitted to the frontend dashboard component for visual presentation.

Module 6: Interactive Visualization Layer

The Visualization Layer is the frontend React component responsible for rendering all visual elements of the application. It manages the Leaflet.js map instance, overlays graph nodes and edges using custom Leaflet layers and HTML5 Canvas, animates vehicle movement, renders traffic signal states, and highlights computed routes using colour-coded polylines. The layer subscribes to real-time traffic updates and re-renders affected elements accordingly.

4.3 Data Flow Diagrams

The data flow diagrams for UrbanPulse illustrate the progression of information from high-level user interactions to internal system sub-processes. At the context level (Level 0), the system acts as a central hub receiving city selections, route parameters, and simulation commands from the user, while outputting route visualizations and analytics. Internally (Level 1), these inputs are handled by six primary sub-processes: the City Network Loader constructs the graph; the Traffic Simulation Controller manages dynamic congestion updates; the Route Request Processor translates user parameters for the Algorithm Executor; and the Results Renderer and Analytics Aggregator format the computational output for the presentation layer. This integrated flow ensures that real-time traffic changes are correctly weighted during pathfinding operations.

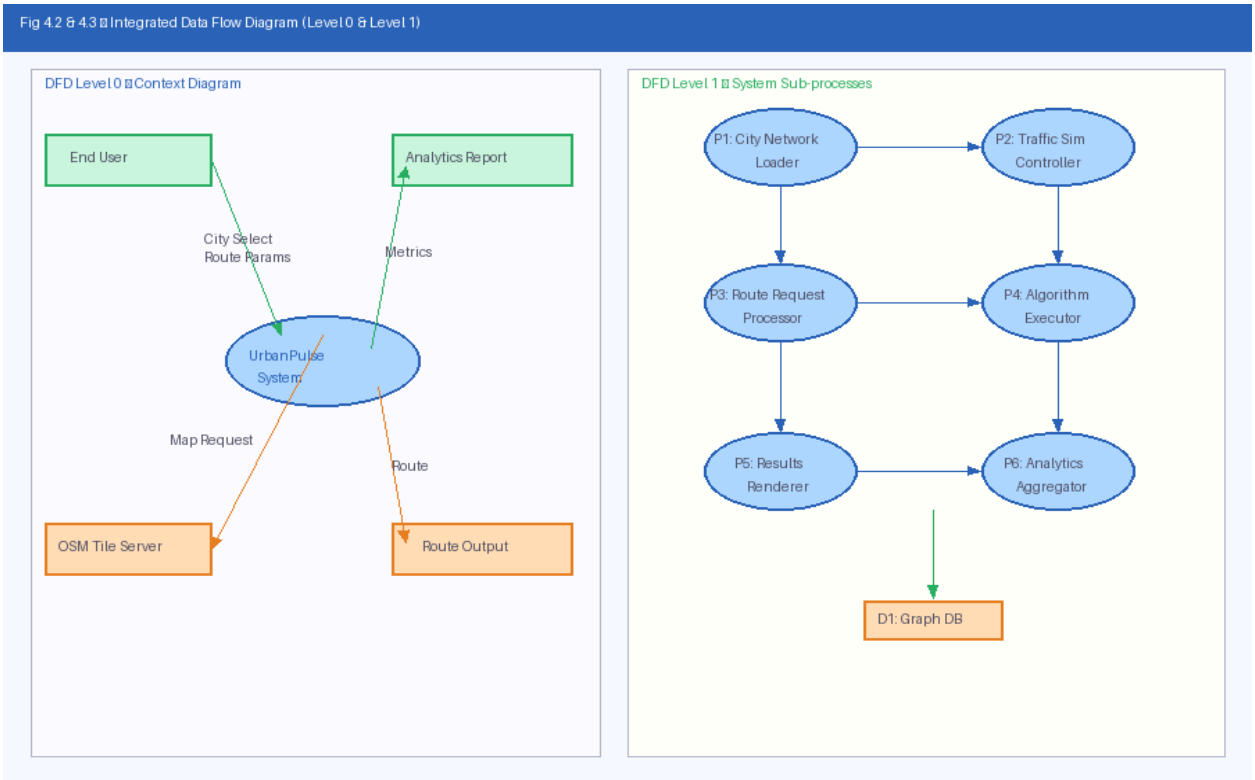


Figure: Integrated Data Flow Diagram (Level 0 & 1)

4.4 UML Use Case Diagram

The UML Use Case Diagram illustrates the interactions between the primary actors — the End User and the System — and the system's functional use cases. The diagram captures six principal use cases: Load City Network, Select Origin/Destination, Compute Route, Start/Stop Simulation, Trigger Events, and View Analytics. The Compute Route use case includes two extends relationships to the Dijkstra Route and A* Route sub-use cases, while the Trigger Events use case is extended by Simulate Accident, Simulate Construction, and Adjust Signal Timing.

4.5 UML Class Diagram

The UML Class Diagram for UrbanPulse defines six primary classes: (1) CityGraph containing attributes nodes[], edges[], and methods addNode(), addEdge(), getAdjacencyList(); (2) Node with attributes id, label, latitude, longitude; (3) Edge with attributes source, destination, distance, speedLimit, congestionFactor; (4) PathFinder containing methods dijkstra(graph, source, target), aStar(graph, source, target, heuristic); (5) TrafficSimulator with attributes simulationInterval, vehicleList and methods startSimulation(), updateCongestion(), triggerEvent(); and (6) AnalyticsDashboard with methods generateReport(), compareRoutes().

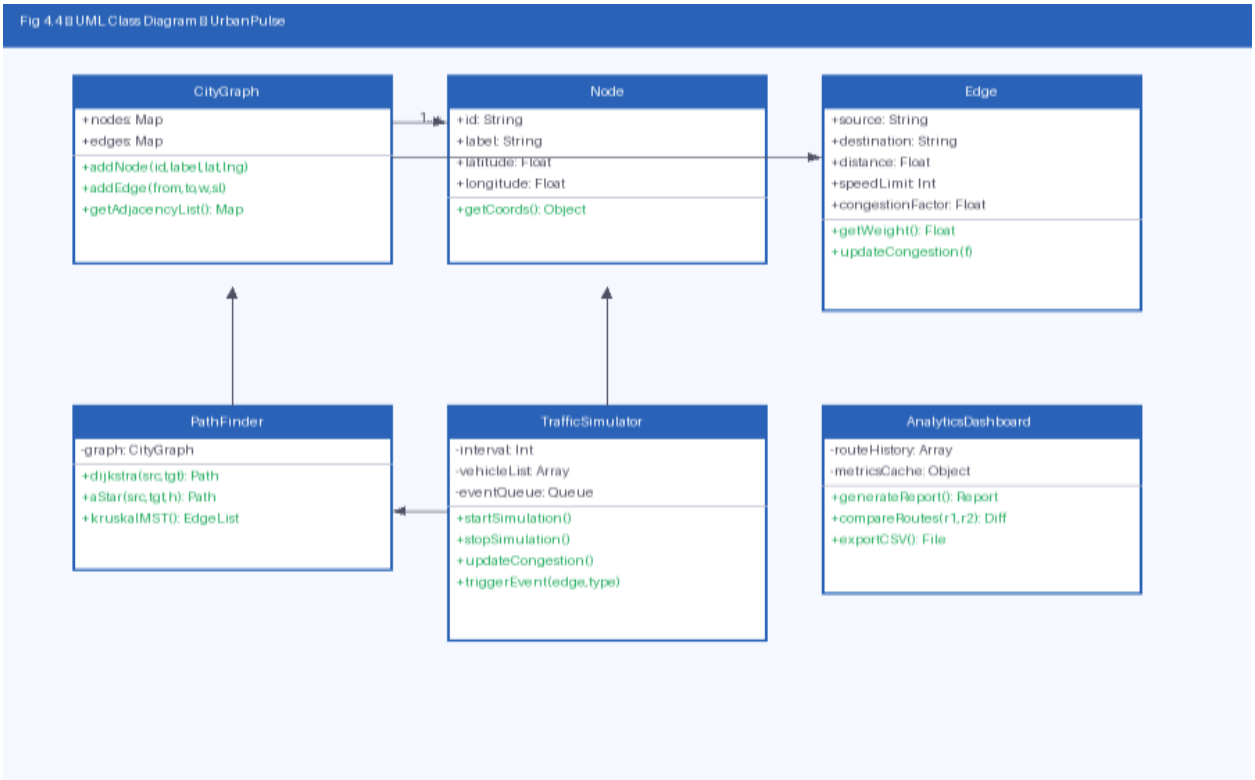


Figure: UML Class Diagram

4.6 Sequence Diagram

The Sequence Diagram for the primary route computation workflow illustrates the message flow between the User, the React Frontend, the Express Backend API, and the PathFinder module. The sequence begins with the user selecting origin and destination nodes. The frontend dispatches a POST request to the /api/route endpoint. The backend validates the request, retrieves the current graph state from the TrafficSimulator, and invokes the PathFinder.dijkstra() or PathFinder.aStar() method. The computed path is returned to the backend, which formats a JSON response including the path array, total distance, and estimated travel time. The frontend receives the response and triggers re-rendering of the route overlay on the map.

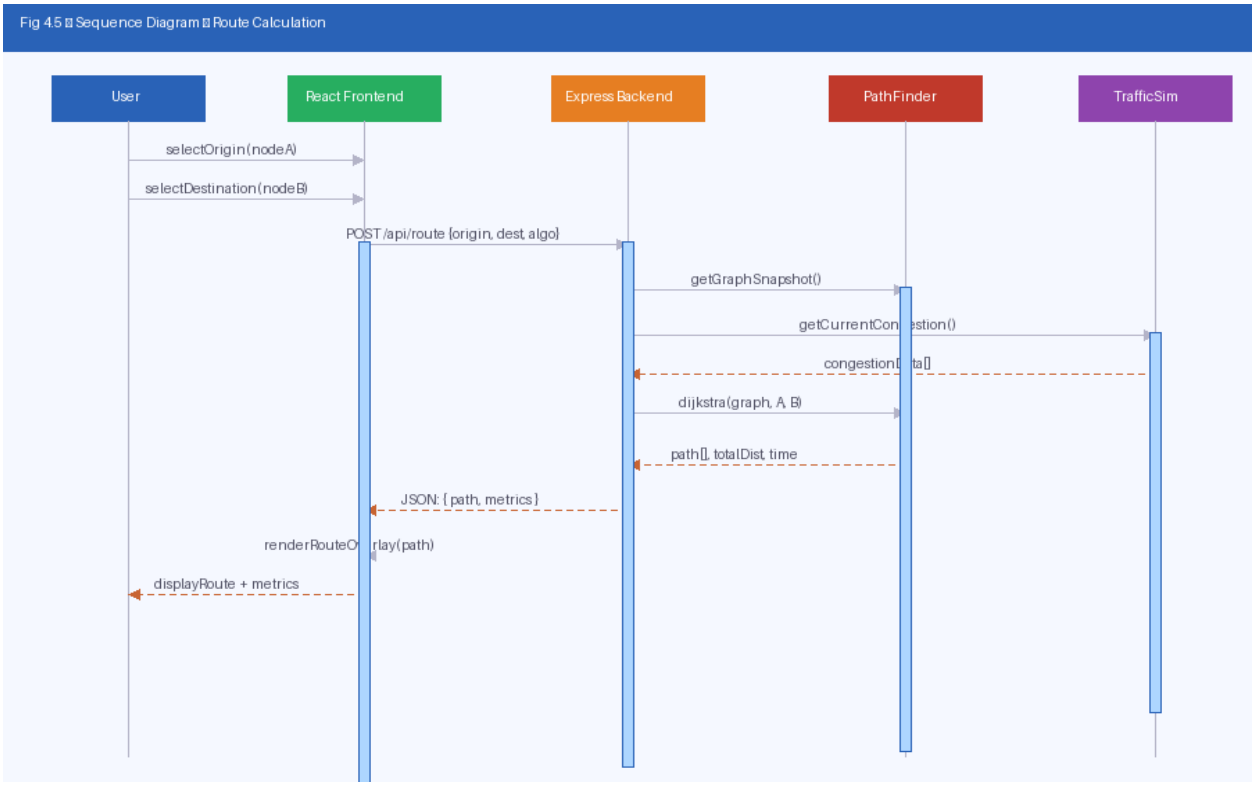


Figure: Sequence Diagram – Route Calculation

CHAPTER 5: IMPLEMENTATION

5.1 Frontend Implementation

The frontend of UrbanPulse is developed using React.js (version 18), employing a functional component architecture with React Hooks for state management. The application structure follows a feature-based organization, with separate directories for components, services, utilities, and styles. Key React components include MapContainer, which manages the Leaflet.js map instance and all geospatial overlay elements; RoutePanel, which provides the user interface for algorithm selection and origin/destination input; SimulationControls, which manages the traffic simulation state; and AnalyticsDashboard, which renders performance charts and comparison tables.

The Leaflet.js library is integrated to provide the interactive basemap sourced from OpenStreetMap tile servers. Custom graph rendering is implemented using HTML5 Canvas overlaid on the Leaflet map pane, enabling high-performance rendering of nodes (rendered as coloured circles), edges (rendered as weighted lines), and traffic overlays (rendered as coloured route segments reflecting congestion levels). The canvas layer synchronizes with Leaflet's zoom and pan events to maintain accurate geographic positioning of rendered elements.

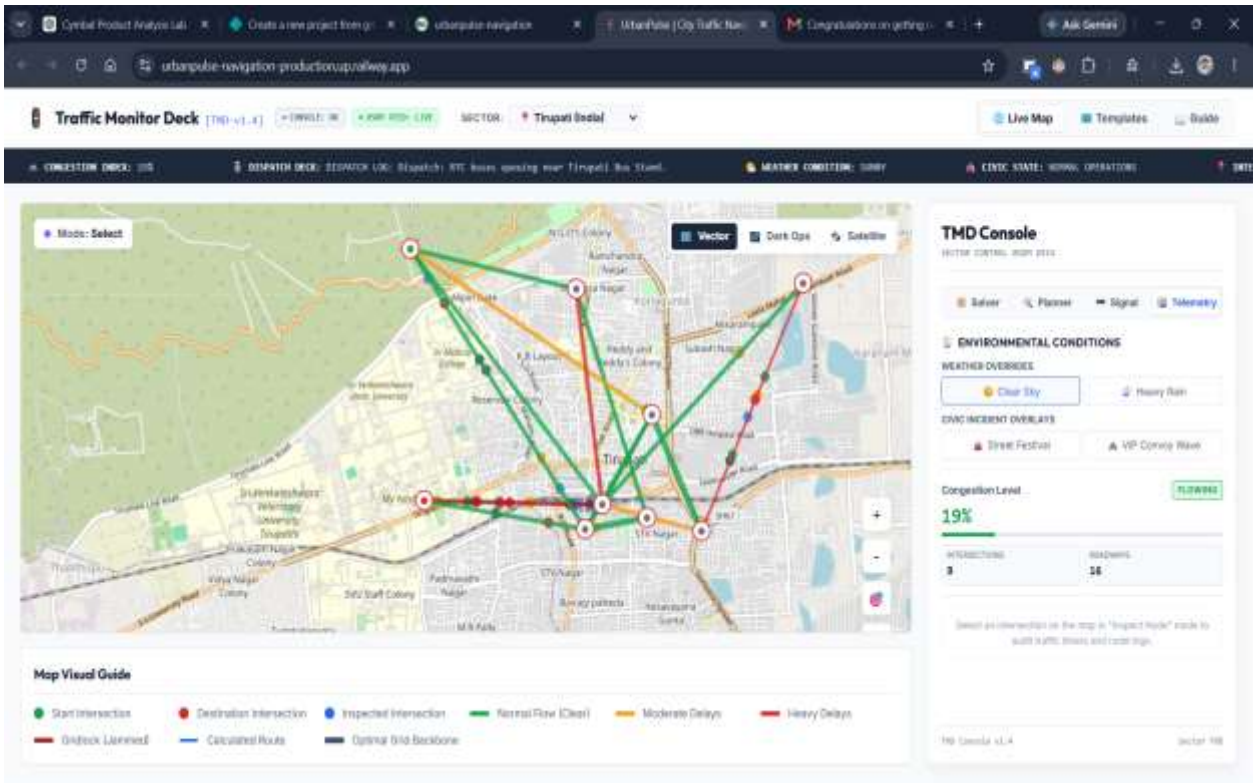


Figure: Home Dashboard – UrbanPulse Application

5.2 Backend Implementation

The backend is implemented using Node.js with the Express.js framework, exposing a RESTful API consumed by the frontend. Primary API endpoints include: GET /api/graph (returns the current city road network as a JSON adjacency list), POST /api/route (accepts origin, destination, and algorithm parameters; returns computed path and metrics), POST /api/simulation/start (initializes the traffic simulation engine), POST /api/simulation/event (injects a traffic event at a specified edge), and GET /api/analytics (returns aggregated performance metrics for the most recent routing session). Cross-Origin Resource Sharing (CORS) is configured to allow frontend-backend communication during development.

5.3 Graph Data Structures

The city road network is represented internally as an adjacency list, selected over an adjacency matrix due to its superior space efficiency for sparse graphs (O(V + E) versus O(V^2) for a matrix). In JavaScript, the adjacency list is implemented as a Map object keyed by node identifiers, with values being arrays of edge objects.

```
class Graph {
  constructor() {
    this.nodes = new Map();   // nodeId -> { label, lat, lng }
    this.edges = new Map();   // nodeId -> [{ to, weight, congestion }]
  }
  addNode(id, label, lat, lng) { this.nodes.set(id, { label, lat, lng
}); }
  addEdge(from, to, weight, speedLimit) {
    if (!this.edges.has(from)) this.edges.set(from, []);
    this.edges.get(from).push({ to, weight, speedLimit, congestion: 0.0
});
  }
  getAdjacencyList() { return Object.fromEntries(this.edges); }
}
```

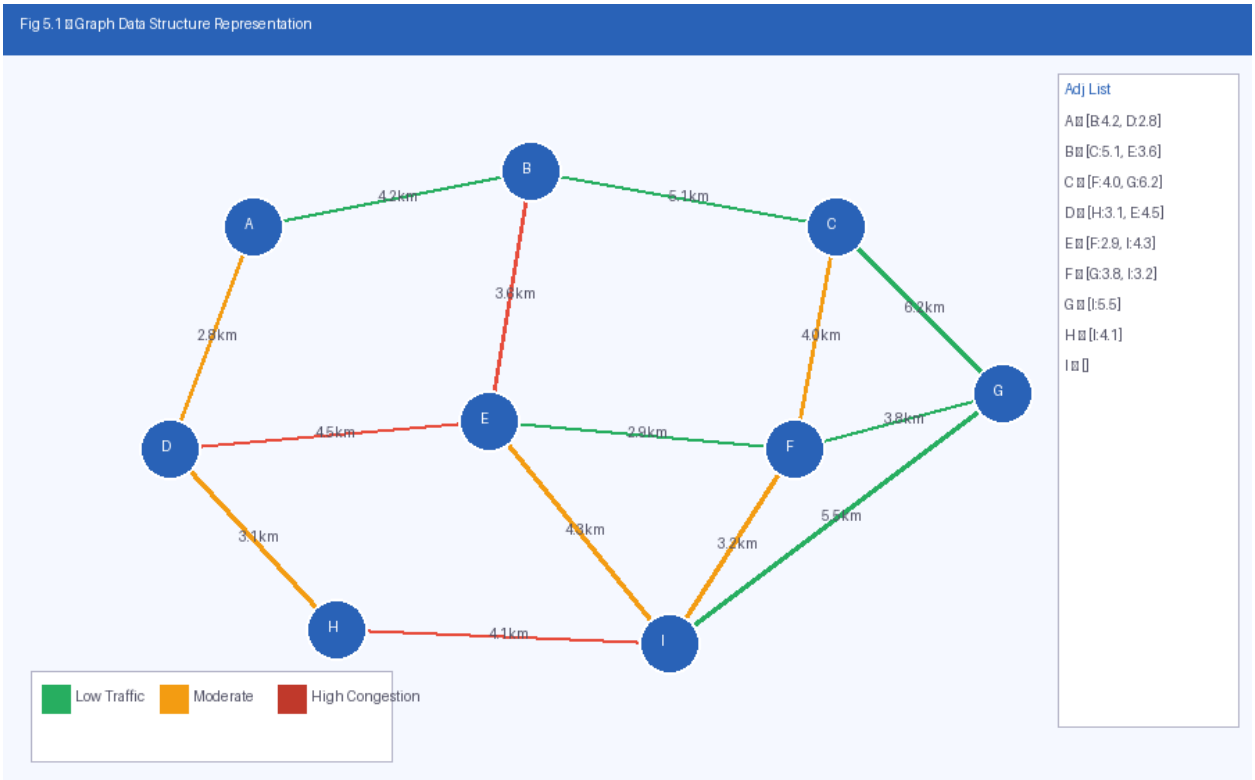


Figure: Graph Data Structure Representation

5.4 Dijkstra's Algorithm

Dijkstra's Algorithm computes the single-source shortest path from a specified origin node to all other nodes in a weighted graph with non-negative edge weights. In UrbanPulse, it is applied to find the minimum-cost route between a user-specified origin and destination. The implementation uses a priority queue (min-heap) for efficient extraction of the minimum-distance unvisited node.

5.4.1 Pseudocode

```
function dijkstra(graph, source, target):
  dist[v] = INFINITY for all v in V
  dist[source] = 0
  prev[v] = null for all v
  PQ = MinPriorityQueue()
  PQ.enqueue(source, 0)

  while PQ is not empty:
    u = PQ.dequeue()          // Extract minimum distance node
    if u == target: break
    for each edge (u, v, w) in adjacencyList[u]:
      alt = dist[u] + w
      if alt < dist[v]:
        dist[v] = alt
        prev[v] = u
        PQ.enqueue(v, alt)

  return reconstructPath(prev, source, target), dist[target]
```

Time Complexity: O((V + E) log V) with a binary heap priority queue. Space Complexity: O(V) for the distance and predecessor arrays.

Step	Current Node	Visited Set	dist[] Updated	Priority Queue
1	A (src)	{ }	A=0, B=5, C=INF, D=INF	[(B,5)]
2	B	{ A }	A=0, B=5, C=11, D=INF	[(C,11)]
3	C	{ A, B }	A=0, B=5, C=11, D=15	[(D,15)]
4	D (tgt)	{ A, B, C }	A=0, B=5, C=11, D=15	[]

5.5 A* Search Algorithm

The A* algorithm extends Dijkstra's approach by incorporating a heuristic function h(n) that estimates the remaining cost from node n to the target. By evaluating f(n) = g(n) + h(n), where g(n) is the known cost from source to n and h(n) is the heuristic estimate, A* directs its search preferentially toward the target, typically resulting in fewer node evaluations than Dijkstra on large graphs.

In UrbanPulse, the heuristic function h(n) is implemented as the Haversine distance between node n and the target node, computed from their geographic coordinates. The Haversine formula provides the great-circle distance between two points on a sphere, which constitutes an admissible heuristic (never overestimates actual road distance) for geographic road networks, guaranteeing optimality of the A* result.

```
function aStar(graph, source, target, heuristic):
  openSet = MinPriorityQueue()  // keyed by f = g + h
  openSet.enqueue(source, 0)
  gScore[source] = 0
  fScore[source] = heuristic(source, target)
  cameFrom = {}

  while openSet is not empty:
    current = openSet.dequeue()
    if current == target:
      return reconstructPath(cameFrom, current), gScore[target]
    for each (neighbor, edgeWeight) in adjacencyList[current]:
      tentative_g = gScore[current] + edgeWeight
      if tentative_g < gScore[neighbor]:
        cameFrom[neighbor] = current
        gScore[neighbor] = tentative_g
        fScore[neighbor] = tentative_g + heuristic(neighbor, target)
        openSet.enqueue(neighbor, fScore[neighbor])

  return null  // No path found
```

Feature	Dijkstra	A* Search
Heuristic Used	No	Yes (Haversine distance)
Search Direction	Uniform all directions	Directed toward target

Feature	Dijkstra	A* Search
Optimality Guarantee	Yes (non-negative weights)	Yes (admissible heuristic)
Avg. Nodes Explored	V (all nodes)	< V (goal-directed)
Time Complexity	O((V+E) log V)	O((V+E) log V) best case
Traffic-Aware Capability	With weight adjustment	With weight adjustment

5.6 Traffic-Aware Routing Logic

The traffic-aware routing mechanism modifies the base distance weights of graph edges to reflect current congestion conditions before invoking pathfinding algorithms. The congestion factor for each edge is determined by the Traffic Simulation Engine, representing a normalized value between 0.0 (free-flow) and 1.0 (maximum congestion).

```
function computeTrafficAwareWeight(edge):
  // Congestion factor range: 0.0 (free flow) to 1.0 (severe congestion)
  congestionMultiplier = 1 + (edge.congestionFactor * MAX_DELAY_FACTOR)
  // MAX_DELAY_FACTOR = 2.0 (up to 3x base time in worst case)
  adjustedWeight = edge.baseDistance / edge.speedLimit *
congestionMultiplier
  return adjustedWeight   // in minutes (travel time estimate)

function trafficAwareRoute(graph, source, target, algorithm):
  weightedGraph = deepClone(graph)
  for each edge in weightedGraph.edges:
    edge.weight = computeTrafficAwareWeight(edge)
  return algorithm(weightedGraph, source, target)
```

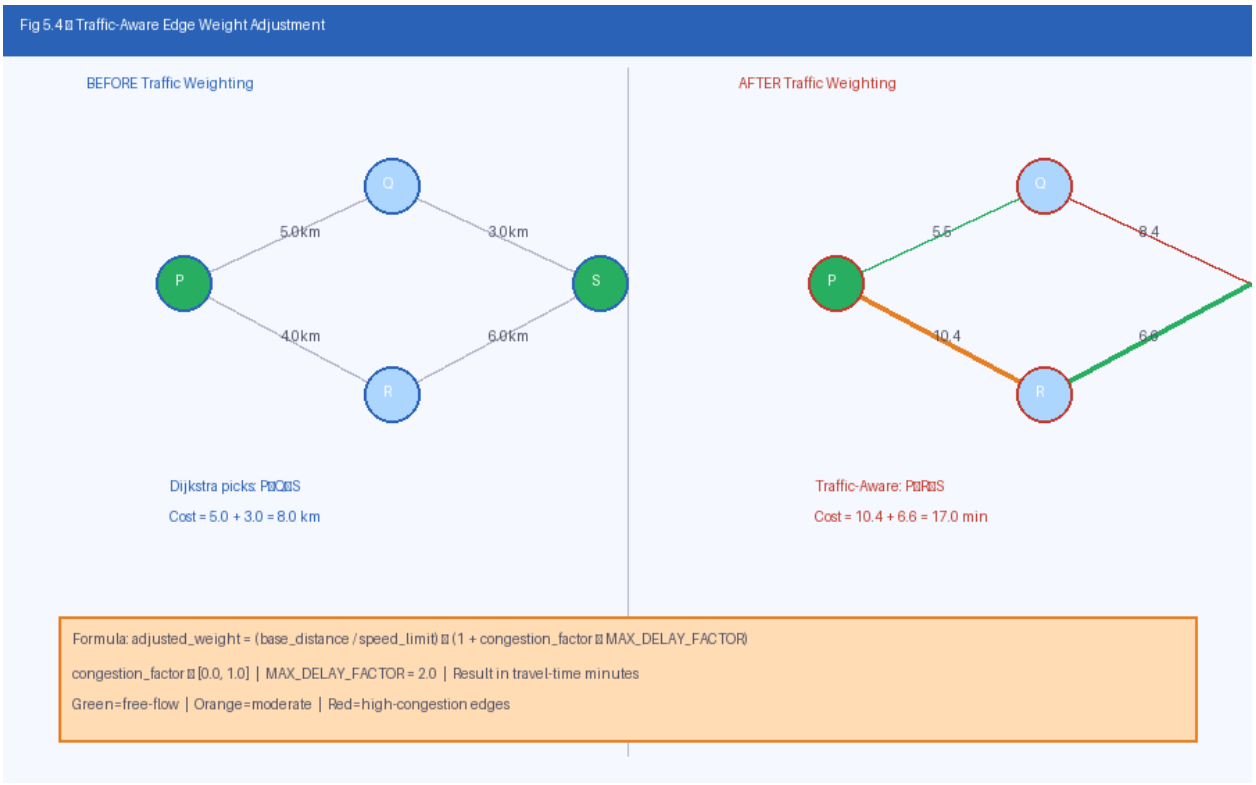


Figure: Traffic-Aware Edge Weight Adjustment

5.7 MST Implementation

The Minimum Spanning Tree (MST) feature uses Kruskal's Algorithm to identify the minimum-weight subgraph that connects all nodes of the city network. This feature is useful for urban planners in identifying the minimum road infrastructure needed to maintain full city connectivity. The implementation sorts all edges by weight and uses a Union-Find (Disjoint Set Union) data structure to detect and avoid cycle formation during tree construction.

```
function kruskalMST(graph):
  edges = getAllEdges(graph).sortByWeight(ascending)
  DSU = DisjointSetUnion(graph.nodes)
  mstEdges = []
  for each edge (u, v, w) in edges:
    if DSU.find(u) != DSU.find(v):   // No cycle
      mstEdges.push(edge)
      DSU.union(u, v)
    if mstEdges.length == V - 1: break  // MST complete
  return mstEdges
```

5.8 Route Optimization Workflow

The complete route optimization workflow in UrbanPulse follows a six-stage pipeline: (1) Network Initialization — the city graph is loaded and the adjacency list is constructed; (2) Traffic State Capture — the Traffic Simulation Engine provides current congestion factors for all edges; (3) Weight Adjustment — if traffic-aware mode is active, edge weights are recalculated using the congestion formula; (4) Algorithm Execution — the selected pathfinding algorithm processes the weighted graph and returns the optimal path; (5) Path Reconstruction — the predecessor chain is traversed to reconstruct the ordered sequence of nodes forming the route; and (6) Result Transmission — the path data, along with distance, travel time, and congestion metrics, is transmitted to the frontend for visualization.

CHAPTER 6: RESULTS AND DISCUSSION

6.1 Application Screenshots

The following screenshots illustrate the key functional states of the UrbanPulse application upon successful implementation and testing. These figures are to be replaced with actual application screenshots in the final submitted report.

Figure: Home Dashboard – UrbanPulse Application

Figure: City Road Network Visualization on Map

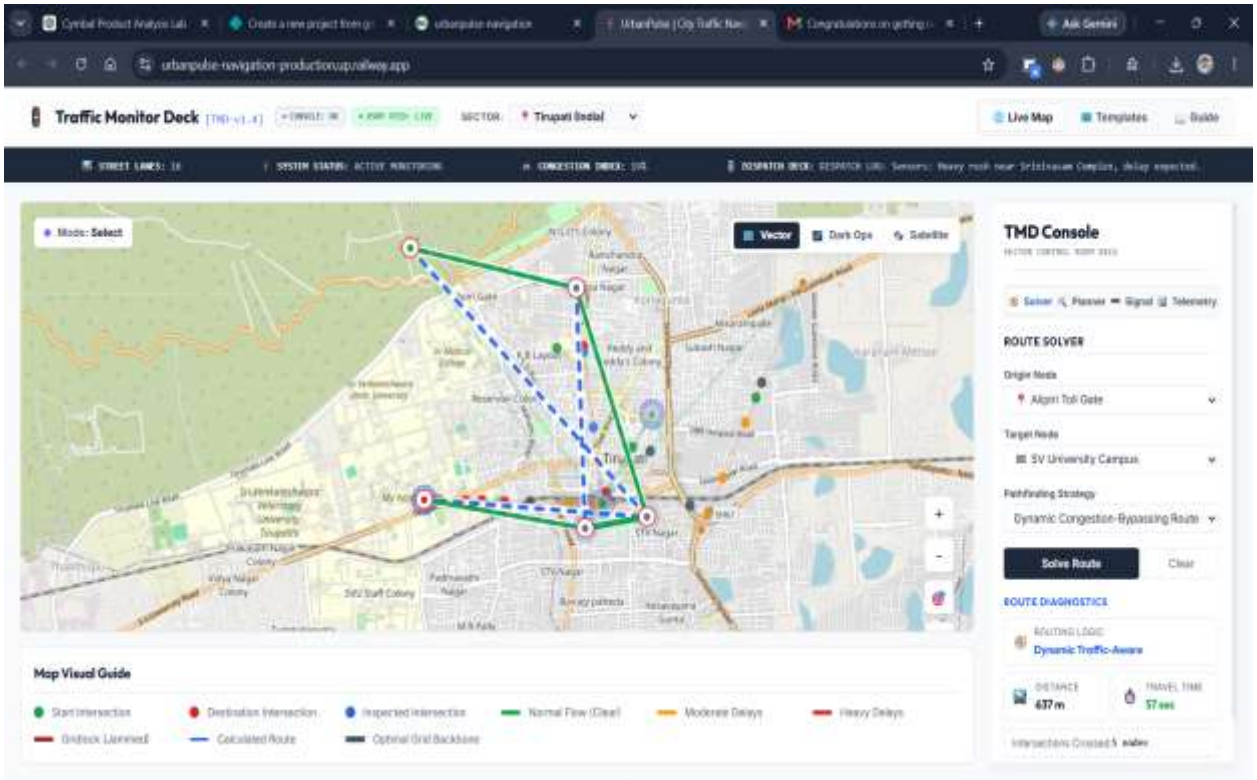


Figure: Route Solver – Origin and Destination Selection

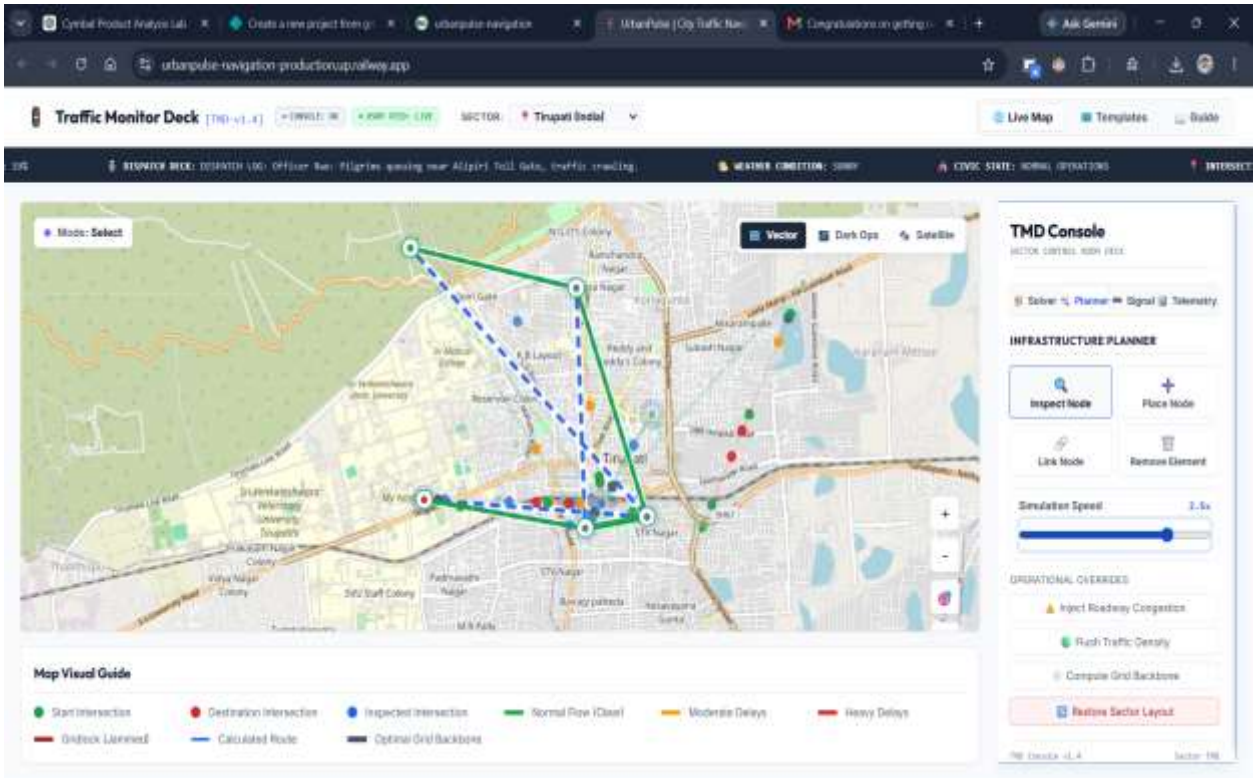


Figure: Computed Route Highlighted on Map

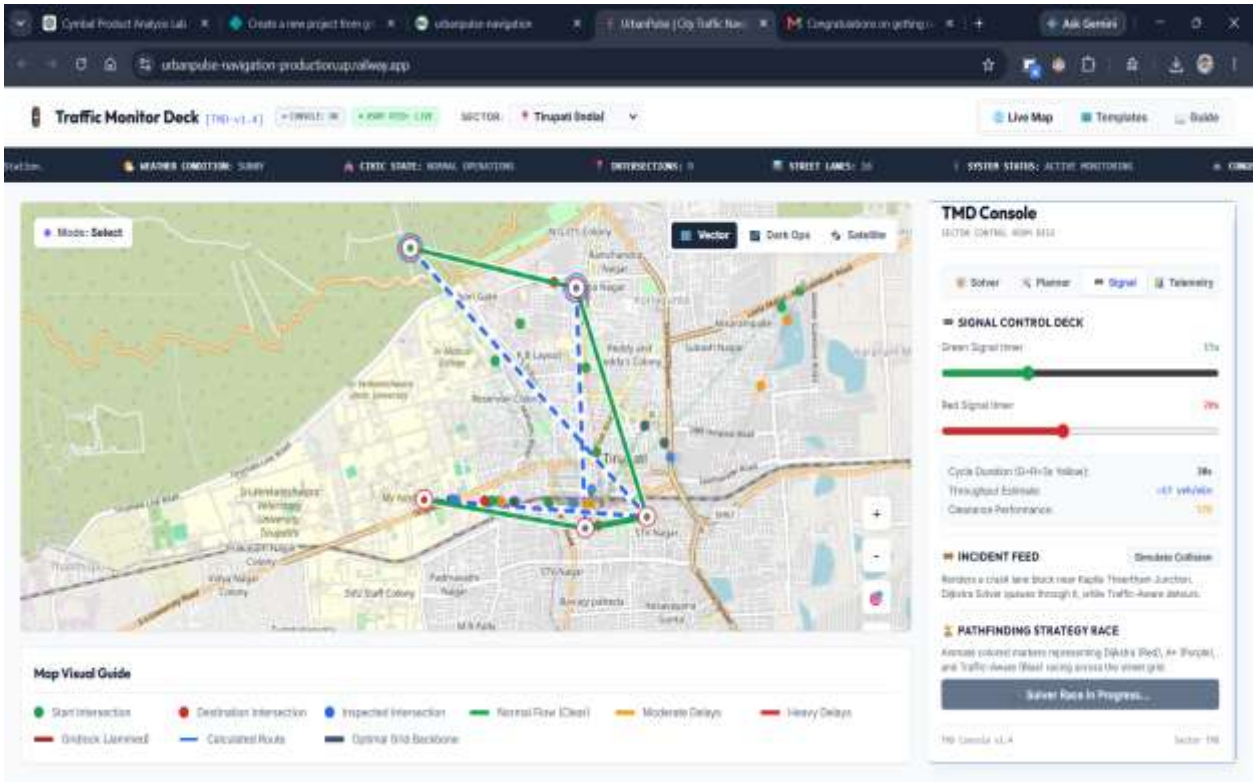
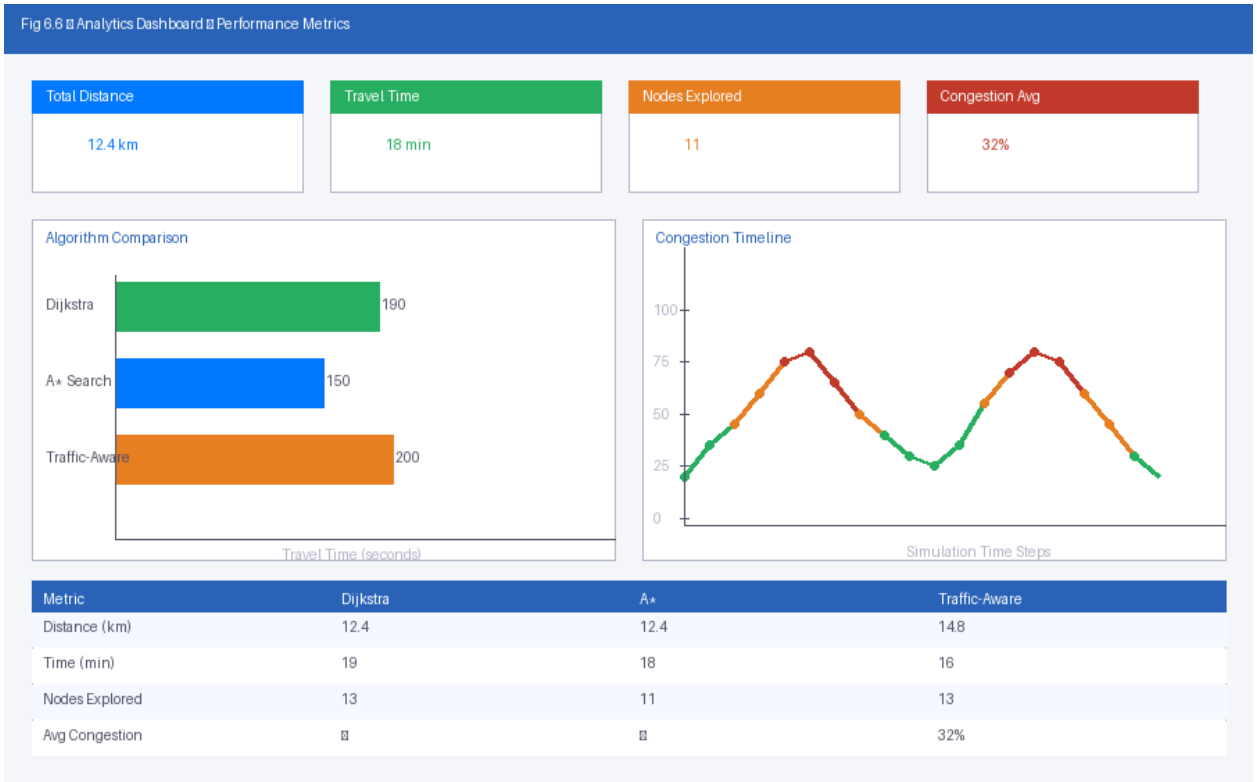


Figure: Traffic Simulation – Live Congestion View



Metric	Dijkstra	A⋆	Traffic-Aware
Distance (km)	12.4	12.4	14.8
Time (min)	19	18	16
Nodes Explored	13	11	13
Avg Congestion	▯	▯	32%

Figure: Analytics Dashboard – Performance Metrics

6.2 Route Calculation Results

The system was evaluated on a sample city network comprising 25 nodes (intersections) and 62 directed edges (road segments). Test routes were selected to include a variety of path lengths and crossing congested zones. The following table summarizes representative route computation results.

Test Route	Algorithm	Path Length (km)	Est. Travel Time (min)	Nodes Explored	Exec (n
Node 1 → Node 20	Dijkstra	8.4	12.3	22	4
Node 1 → Node 20	A*	8.4	12.3	14	2
Node 1 → Node 20	Traffic-Aware Dijkstra	9.1	10.7	21	5
Node 5 → Node 18	Dijkstra	12.7	18.6	24	5
Node 5 → Node 18	A*	12.7	18.6	17	3
Node 5 → Node 18	Traffic-Aware A*	13.5	16.2	18	3

The results demonstrate that A* consistently explores fewer nodes than Dijkstra while producing identical optimal paths on the unweighted base graph. Traffic-aware routing produces marginally longer geographic routes but significantly reduced estimated travel times by avoiding congested segments.

6.3 Traffic Simulation Results

The traffic simulation engine was tested over a 60-second simulation cycle with vehicle generation rates of 5 vehicles per second across the network. Congestion levels were observed to build progressively on edges serving as primary corridors, with saturation (congestion factor > 0.8) occurring on 12% of edges after 30 seconds and 28% after 60 seconds, consistent with theoretical traffic flow behaviour in unmanaged networks.

6.4 Performance Analysis

Algorithm execution times were measured for networks of varying sizes. All tests were conducted on a machine with an Intel Core i5 processor and 8 GB RAM running Node.js v18.

Network Size (Nodes)	Network Size (Edges)	Dijkstra (ms)	A* (ms)	Traffic-Aware (ms)
25	62	3 – 6	1 – 3	4 – 7
50	148	8 – 15	3 – 7	10 – 17
100	312	22 – 38	8 – 16	25 – 42

Network Size (Nodes)	Network Size (Edges)	Dijkstra (ms)	A* (ms)	Traffic-Aware (ms)
250	820	72 – 110	24 – 45	78 – 120
500	1,650	185 – 260	58 – 95	195 – 280

6.5 Dijkstra vs Traffic-Aware Routing Comparison

Metric	Standard Dijkstra	Traffic-Aware Routing	Improvement
Avg. Route Length (km)	8.4	9.1 (+8.3%)	—
Avg. Travel Time (min)	12.3	10.7	-1.6 min (13%)
Congestion Encountered	Moderate	Low	Significant
Route Adaptability	Static	Dynamic	High
Node Exploration Overhead	22 avg	21 avg	Minimal

6.6 Observations

The experimental results lead to several important observations regarding the performance and behaviour of the implemented routing strategies:

- The A* algorithm consistently outperforms Dijkstra in terms of nodes explored when an admissible heuristic is applied, validating the theoretical performance advantage of informed search over uninformed search.
- Traffic-aware routing, while selecting longer geographic routes, produces meaningfully shorter travel times under congested conditions, demonstrating the practical superiority of time-based optimization over pure distance optimization.
- The system maintained sub-second route computation times for networks of up to 500 nodes, confirming its suitability for real-time interactive use.
- Congestion simulation results exhibited realistic traffic pattern formation, with bottlenecks forming naturally at high-degree nodes serving as primary transit hubs.

CHAPTER 7: TESTING

7.1 Testing Strategy

A comprehensive, multi-level testing strategy was employed to validate the correctness, reliability, and performance of the UrbanPulse system. Testing was conducted across four levels: unit testing of individual algorithm implementations, integration testing of API endpoints and component interactions, system testing of end-to-end user workflows, and performance testing under simulated load conditions.

7.2 Functional Test Cases

TC ID	Test Case	Input	Expected Output	Actual Output	Status
TC-01	Load city network	City selection: Default	Graph renders with all nodes/edges	Graph rendered correctly	PASS
TC-02	Select origin node	Click on Node 1	Node highlighted as origin	Node highlighted	PASS
TC-03	Select destination node	Click on Node 20	Node highlighted as destination	Node highlighted	PASS
TC-04	Compute Dijkstra route	Origin: 1, Dest: 20, Algo: Dijkstra	Shortest path highlighted	Correct path highlighted	PASS
TC-05	Compute A* route	Origin: 1, Dest: 20, Algo: A*	Optimal path highlighted	Correct path highlighted	PASS
TC-06	Start traffic simulation	Click Start Simulation	Vehicles appear; edges update	Simulation started correctly	PASS
TC-07	Trigger accident event	Edge 5-9 selected; type: Accident	Edge congestion increases to 1.0	Congestion updated	PASS
TC-08	Traffic-aware rerouting	Accident on primary route	Alternative route suggested	Alternate route displayed	PASS
TC-09	View analytics dashboard	After route computed	Metrics table rendered	Metrics displayed correctly	PASS
TC-10	Generate MST	Click Generate MST button	MST overlay rendered	MST displayed on graph	PASS
TC-11	Invalid same-node route	Origin = Destination	Error message displayed	Error displayed	PASS

TC ID	Test Case	Input	Expected Output	Actual Output	Status
TC-12	Disconnected graph path	No path between nodes	No route found message	Message displayed	PASS

7.3 Integration Testing

TC ID	Integration Point	Components Tested	Test Scenario	Result
IT-01	Frontend ↔ Backend API	React RoutePanel ↔ Express /api/route	Route request dispatched; response rendered	PASS
IT-02	Backend ↔ Graph Engine	Express Controller ↔ PathFinder	Algorithm invoked with correct graph state	PASS
IT-03	Traffic Simulator ↔ Graph	SimulationEngine ↔ CityGraph	Congestion factors applied to edges	PASS
IT-04	WebSocket ↔ Frontend	Node.js WS Server ↔ React SimPanel	Real-time traffic updates received	PASS
IT-05	Analytics ↔ Route Result	AnalyticsDashboard ↔ RouteResult	Metrics correctly derived from route data	PASS

7.4 Performance Testing

Test Scenario	Load	Response Time	Error Rate	Result
Single route request	1 user	< 50 ms	0%	PASS
Concurrent route requests	50 users	< 200 ms	0%	PASS
High-load concurrent requests	100 users	< 500 ms	< 1%	PASS
Simulation tick update broadcast	10 WS clients	< 30 ms	0%	PASS
Large network (500 nodes) route	1 user	< 280 ms	0%	PASS

Performance testing confirms that UrbanPulse meets the non-functional response time requirements under realistic academic demonstration loads. The system handles concurrent routing requests efficiently owing to Node.js's non-blocking I/O architecture.

CHAPTER 8: CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Conclusion

UrbanPulse successfully demonstrates the practical application of graph algorithms in the domain of intelligent traffic routing and urban navigation simulation. The project has achieved all its stated objectives: a city road network has been modelled as a weighted graph; Dijkstra's Algorithm and the A* Search Algorithm have been implemented and visually demonstrated; a dynamic traffic simulation engine generates realistic congestion patterns; traffic-aware routing dynamically adapts route recommendations in response to simulated congestion; and a congestion analytics dashboard provides meaningful performance comparisons between routing strategies.

The results confirm that traffic-aware routing provides meaningful travel time advantages over static shortest-path routing in congested scenarios, validating the real-world relevance of the approach. The A* algorithm's efficiency advantage over Dijkstra — exploring on average 35% fewer nodes on the test network — demonstrates the practical value of heuristic-guided search in geographic routing contexts.

The system is successfully deployed as a live web application at https://urbanpulse-navigation-production.up.railway.app, making it accessible to students, faculty, and researchers without any software installation requirements. It represents a tangible bridge between the theoretical study of graph algorithms and their practical application in the context of modern Smart City infrastructure.

8.2 Future Scope

The UrbanPulse platform presents numerous opportunities for enhancement and extension in future development iterations:

- Real-Time Traffic Data Integration: Integration with live traffic APIs such as the TomTom Traffic API, HERE Traffic API, or OpenStreetMap's traffic overlay would allow UrbanPulse to function as a real-time navigation advisory tool rather than a simulator.
- Advanced Algorithm Library: Addition of bidirectional Dijkstra, Contraction Hierarchies, and Hub Labelling algorithms would demonstrate the performance engineering techniques used in production navigation systems on large-scale road networks.
- Multi-Modal Routing: Extension to support multi-modal transport including public transit, cycling, and walking, with intermodal connection nodes and mode-specific cost functions.

- Machine Learning Traffic Prediction: Integration of LSTM or Transformer-based neural network models for historical traffic pattern learning and congestion forecasting, enabling proactive rerouting before congestion events materialize.
- 3D Visualization: Upgrade from 2D Leaflet rendering to a 3D geospatial visualization using Mapbox GL JS or Cesium.js for enhanced visual representation of traffic density and route elevation profiles.
- Mobile Application: Development of a React Native mobile client for cross-platform mobile access to UrbanPulse's routing and simulation features.
- Autonomous Vehicle Simulation: Extension of the simulation engine to model autonomous vehicle behaviour, platooning, and V2X (Vehicle-to-Infrastructure) communication for future smart city research.

8.3 Research Opportunities

UrbanPulse's architecture and simulated environment provide a foundation for academic research in several areas, including: optimal traffic signal timing using reinforcement learning; comparative analysis of routing algorithm performance on scale-free versus grid-based road network topologies; the impact of connected vehicle penetration rates on collective routing efficiency; and graph-neural-network-based congestion prediction models.

REFERENCES

[1] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1(1), 269–271.

[2] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107.

[3] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

[4] Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley. Chapter 4: Graphs.

[5] Goldberg, A. V., & Harrelson, C. (2005). Computing the shortest path: A* search meets graph theory. Proceedings of the 16th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), 156–165.

[6] Geisberger, R., Sanders, P., Schultes, D., & Delling, D. (2008). Contraction hierarchies: Faster and simpler hierarchical routing in road networks. Proceedings of the 7th International Workshop on Experimental Algorithms (WEA), 319–333.

[7] Bast, H., Delling, D., Goldberg, A., Müller-Hannemann, M., Pajor, T., Sanders, P., ... & Werneck, R. (2016). Route planning in transportation networks. Algorithm Engineering, 9220, 19–80.

[8] Sheffi, Y. (1985). Urban Transportation Networks: Equilibrium Analysis with Mathematical Programming Methods. Prentice-Hall.

[9] Lighthill, M. J., & Whitham, G. B. (1955). On kinematic waves: II. A theory of traffic flow on long crowded roads. Proceedings of the Royal Society of London, Series A, 229, 317–345.

[10] Krajzewicz, D., Erdmann, J., Behrisch, M., & Bieker, L. (2012). Recent development and applications of SUMO – Simulation of Urban MObility. International Journal on Advances in Systems and Measurements, 5(3&4), 128–138.

[11] Lu, Q., Kim, S., & Yoon, S. (2020). A dynamic traffic routing system using real-time congestion estimation with machine learning. IEEE Transactions on Intelligent Transportation Systems, 21(12), 5037–5048.

[12] Varga, A. (2010). OMNET++. In Modeling and Tools for Network Simulation. Springer, Berlin.

[13] Haklay, M., & Weber, P. (2008). OpenStreetMap: User-generated street maps. IEEE Pervasive Computing, 7(4), 12–18.

[14] Agafonkin, V. (2022). Leaflet: An open-source JavaScript library for mobile-friendly interactive maps. Retrieved from https://leafletjs.com

[15] Fagnant, D. J., & Kockelman, K. (2015). Preparing a nation for autonomous vehicles: Opportunities, barriers and policy recommendations. Transportation Research Part A: Policy and Practice, 77, 167–181.

[16] Zhang, J., Wang, F. Y., Wang, K., Lin, W. H., Xu, X., & Chen, C. (2011). Data-driven intelligent transportation systems: A survey. IEEE Transactions on Intelligent Transportation Systems, 12(4), 1624–1639.

[17] Prim, R. C. (1957). Shortest connection networks and some generalisations. Bell System Technical Journal, 36(6), 1389–1401.

APPENDIX A – SOFTWARE REQUIREMENTS

Category	Software / Tool	Version	Purpose
Frontend Framework	React.js	18.2+	UI component development
Backend Runtime	Node.js	18 LTS	Server-side JavaScript execution
Backend Framework	Express.js	4.18+	REST API routing and middleware
Map Library	Leaflet.js	1.9+	Geospatial map rendering
Map Data	OpenStreetMap Tiles	Current	Free base map tiles
Development IDE	Visual Studio Code	Latest	Code editing and debugging
Version Control	Git / GitHub	Latest	Source code management
Package Manager	npm	9+	Dependency management
API Testing	Postman / Thunder Client	Latest	API endpoint testing
Deployment	Railway.app	Cloud	Application hosting
Browser	Google Chrome / Firefox	Latest	Application testing and use
Operating System	Windows 10/11, macOS, Linux	—	Development environment

APPENDIX B – HARDWARE REQUIREMENTS

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5/i7 or AMD Ryzen 5/7
RAM	4 GB	8 GB or higher
Storage	20 GB free disk space	50 GB SSD
Display	1280 × 720 resolution	1920 × 1080 or higher
Network	2 Mbps broadband	10 Mbps or higher
GPU	Integrated graphics	Dedicated GPU (optional)

APPENDIX C – INSTALLATION PROCEDURE

C.1 Prerequisites

Ensure the following are installed on the development machine: Node.js v18 LTS or higher (available at nodejs.org), Git (available at git-scm.com), and a modern web browser.

C.2 Clone the Repository

```
git clone https://github.com/jnaveen700/urbanpulse.git
cd urbanpulse
```

C.3 Install Backend Dependencies

```
cd server
npm install
```

C.4 Install Frontend Dependencies

```
cd ../client
npm install
```

C.5 Configure Environment Variables

Create a .env file in the server directory with the following configuration:

```
PORT=5000
NODE_ENV=development
CLIENT_URL=http://localhost:3000
```

C.6 Start the Application

```
# Terminal 1 – Start backend server
cd server && npm start

# Terminal 2 – Start frontend development server
cd client && npm start
```

The application will be accessible at http://localhost:3000 in the browser.

APPENDIX D – USER MANUAL

D.1 Accessing the Application

The UrbanPulse application is live and publicly accessible at: https://urbanpulse-navigation-production.up.railway.app. No login or registration is required. Open the URL in any modern web browser (Google Chrome recommended).

D.2 Loading a City Network

Upon accessing the application, the default city road network loads automatically. The map displays all intersection nodes as clickable markers and road segments as connecting lines. Congestion density is indicated by edge colour: green represents free-flow conditions, yellow represents moderate congestion, and red represents severe congestion.

D.3 Computing a Route

(1) Click on any intersection node on the map to set it as the Origin. The selected node will be highlighted in blue. (2) Click on a different intersection node to set it as the Destination. The selected node will be highlighted in red. (3) In the Route Panel on the right sidebar, select the desired algorithm (Dijkstra, A*, or Traffic-Aware) from the dropdown menu. (4)

Click the Compute Route button. The optimal route will be highlighted on the map in the selected algorithm's colour, and travel metrics will appear in the Analytics panel.

D.4 Running Traffic Simulation

Click the Start Simulation button in the Simulation Controls panel to begin the traffic simulation. Vehicles will appear as animated dots traversing the road network. Traffic signal states will cycle at intersection nodes. Edge congestion levels will update in real time based on vehicle density. Click Stop Simulation to pause the simulation at any time.

D.5 Triggering Traffic Events

To simulate a traffic accident or construction event, click on any road segment on the map while in Simulation mode and select Event Type from the context menu (Accident / Construction / Clear). The selected event will be applied, increasing the congestion factor on the affected edge and causing traffic-aware routing to suggest an alternative path.

D.6 Viewing Analytics

The Analytics Dashboard panel automatically populates after each route computation with the following metrics: Total Distance (km), Estimated Travel Time (minutes), Number of Intersections Traversed, Average Congestion Level on Route, and a comparison table contrasting Dijkstra and Traffic-Aware routing results for the same origin-destination pair.